Instituto Tecnológico de Costa Rica (TEC)

Escuela de Ingeniería en Computación

Tarea Programada #1

Sistema de formateo y análisis de datos de países y monedas

Curso	Taller de Programación
Profesora	Laura Coto
Estudiante	Alexandra De Gracia — carné 2026801601
Encargada de calidad	Alexandra De Gracia
Encargada de desarrollo	Alexandra De Gracia
Tarea programada	N.º 1
Semestre y año lectivo	II Semestre 2026
Fecha de entrega	Viernes 16 de octubre de 2026, 11:40 pm (16/10/2026)
Estado de la entrega	**Excelente**
Repositorio de GitHub	https://github.com/AlexandraDeGracia/TP1-FormateoPaisesMonedas-2026

Índice

Cada entrada es un hipervínculo a su sección.

1. Enunciado del proyecto
2. Justificación de eliminación de olores de software
    - Olor 1: código muerto
    - Olor 2: lógica de formateo numérico duplicada
    - Olor 3: método largo en el generador HTML
    - Olor 4: nombres inconsistentes
    - Olor 5: bucles de clasificación duplicados
3. Lista de validaciones estratégicas
4. Estrategia de generación de los reportes
    - Estrategia general
    - Reporte TXT de países
    - Reporte HTML de monedas
    - Reporte HTML de densidad poblacional
5. Reglamento de trabajo
6. Agendas
7. Minutas
8. Cronograma
9. Conclusiones del trabajo
    - Problemas encontrados y soluciones
    - Aprendizajes obtenidos
    - Conclusión del trabajo
10. Estadística de tiempos (PSP)

1. Enunciado del proyecto

El proyecto corresponde a la **Tarea Programada #1** del curso Taller de Programación (II Semestre 2026), titulada «Sistema de formateo y análisis de datos de países y monedas». El enunciado oficial (*public_tareas-programadas_TP1_2S_2026_-_FormateoPaisesMonedas-Rev2_3.pdf*) se adjunta a esta entrega junto con la documentación y el programa fuente. El código y el historial de versiones están en el repositorio de GitHub: https://github.com/AlexandraDeGracia/TP1-FormateoPaisesMonedas-2026.

El sistema simula el consumo de una API pública mediante archivos de texto: lee una base de datos CSV de países y tipos de cambio, normaliza cadenas, realiza cálculos numéricos y genera reportes en TXT, HTML y TOON, manteniendo una bitácora binaria de todas las operaciones.

Restricciones principales

- Python 3.10+, solo modo consola, con menú interactivo de 9 opciones.
- Prohibido usar diccionarios y `.replace()`; las operaciones de cadenas se hacen carácter por carácter.
- Las operaciones numéricas (suma, promedio, mediana, ordenamiento, redondeo) se implementan manualmente, sin `statistics` ni `numpy`.
- Persistencia en TOON, reportes en TXT y HTML, y bitácora en el archivo binario `bitacora.bin`.
- Control de versiones en GitHub y entrega vía TEC Digital el viernes 16 de octubre a las 11:40 pm.

Arquitectura de la solución

Módulo	Responsabilidad
`main.py`	Menú interactivo, entrada de datos del usuario y flujo de las 9 opciones.
`datos.py`	Lectura y validación del CSV; lectura y escritura de `paises.toon` y `tasas_cambio.toon`.
`formateo_strings.py`	Funciones de cadenas carácter por carácter (mayúsculas, tildes, paréntesis, guiones bajos, capitalización).
`calculos.py`	Estadística manual: población, mediana, área, densidad, categorías y tasas de cambio.
`formatos.py`	Redondeo y formato manual de números a N decimales (sin `round()`).
`reportes.py`	Generación de los reportes TXT y HTML.
`bitacora.py`	Registro binario (`pickle`), búsqueda por fecha y palabra clave, y exportación a CSV.

2. Justificación de eliminación de olores de software

Se identificaron y eliminaron cinco *code smells* del código fuente. Cada uno se justifica con su ubicación, el problema que causaba y la corrección aplicada.

Olor 1: código muerto (*dead code*)

Ubicación: `reportes.py`, función `construirTablaDetalladaMonedas`. Una asignación residual a la variable `html` (un literal sin sentido marcado como «placeholder») se sobrescribía de inmediato, por lo que nunca se usaba y ensuciaba el generador. Se eliminó en el commit `713c9cf`.

```
# ANTES
html = "自由" # placeholder replaced below
html = "<h2>Tabla Detallada de Monedas</h2>\n"

# DESPUÉS (commit 713c9cf)
html = "<h2>Tabla Detallada de Monedas</h2>\n"
```

Olor 2: lógica de formateo numérico duplicada

Ubicación: `reportes.py` y `calculos.py`. Como `round()` y el formato automático están prohibidos, el redondeo y el relleno de ceros se habían escrito dos veces, casi idénticos, para 2 y para 4 decimales (`formatear_2_decimales` y `formatear_4_decimales`). Se centralizó la lógica en una sola función, `formatearNDecimales` (`formatos.py`), y las demás pasaron a ser envoltorios de una línea.

```
# ANTES (reportes.py): el mismo bloque repetido, solo cambia el 2 por el 4
def formatear_4_decimales(numero):
    val = calc.redondear_manual(numero, 4)
    partes = fs.separar_cadena(str(val), ".")
    ...
    while len(dec) < 4:
        dec = dec + "0"
    ...

# DESPUÉS (formatos.py)
def formatearNDecimales(numero, decimales):
    """Formatea un número con N decimales exactos evitando notación científica."""
    if decimales < 0:
        decimales = 0

    signo = ""
    if numero < 0.0:
        signo = "-"
        numeroPositivo = -numero
    else:
        numeroPositivo = numero

    factor = 1.0
    i = 0
    while i < decimales:
        factor = factor * 10.0
        i = i + 1

    escalado = int(numeroPositivo * factor + 0.5)
    enteroFactor = int(factor)

    if enteroFactor > 0:
        parteEntera = escalado // enteroFactor
        parteDecimal = escalado % enteroFactor
    else:
        parteEntera = escalado
        parteDecimal = 0

    if decimales == 0:
        return signo + str(parteEntera)

    decStr = str(parteDecimal)
    while len(decStr) < decimales:
        decStr = "0" + decStr

    return signo + str(parteEntera) + "." + decStr

def formatear2Decimales(numero):
    """Formatea un número flotante con exactamente 2 decimales sin notación científica."""
    return formatearNDecimales(numero, 2)

def formatear4Decimales(numero):
    """Formatea un número flotante con exactamente 4 decimales sin notación científica."""
    return formatearNDecimales(numero, 4)
```

Olor 3: método largo en el generador HTML

Ubicación: `reportes.py`. La generación del reporte de monedas mezclaba estadísticas, tablas y estilos en un solo bloque. Se dividió en componentes de una sola responsabilidad: `construirCardEstadisticasMonedas`, `construirTablaDetalladaMonedas` y `construirTablaClasificacionMonedas`. Ahora cada parte se puede probar y modificar por separado, y la función principal solo concatena los bloques.

```
# DESPUÉS (reportes.generarReporteMonedasHtml, fragmento)
    html = html + construirCardEstadisticasMonedas(totalMonedas, promedioTasa, monedaFuerte,
monedaDebil, stCard, stP)
    html = html + construirTablaDetalladaMonedas(monedasUnicas, stTable, stThL, stThC, stFuerte,
stDebil, stIgual)
    html = html + construirTablaClasificacionMonedas(mayores1, iguales1, menores1, codigosFuertes,
codigosIguales,
                                                     codigosDebiles, stTable, stThC, stFuerte,
stDebil, stIgual)
    html = html + "</body>\n</html>\n"
```

Olor 4: nombres inconsistentes

Ubicación: funciones de todos los módulos (hoy 112 funciones). Había una mezcla de `snake_case`, prefijos de guion bajo y nombres poco descriptivos. Se estandarizaron a `camelCase` con nombres que explican su propósito (commit `9b3c3ac`).

Antes	Después
`generar_reporte_paises_txt`	`generarReportePaisesTxt`
`_construir_card_estadisticas_monedas`	`construirCardEstadisticasMonedas`
`obtener_codigos_por_clasificacion`	`obtenerCodigosPorClasificacion`
`generar_grafico_ascii_vertical`	`generarGraficoAsciiVertical`
`redondear_manual`	`formatearNDecimales` (`formatos.py`)

Olor 5: bucles de clasificación duplicados

Ubicación: `reportes.py`. Listar los códigos de las monedas «fuertes» (tasa > 1), «iguales» (= 1) y «débiles» (< 1) requeriría tres bucles casi idénticos. Se unificó en la función parametrizada `obtenerCodigosPorClasificacion`, que recibe la condición buscada y devuelve los códigos correspondientes.

```
def obtenerCodigosPorClasificacion(listaMonedas, condicion):
    """Retorna una cadena con los códigos de monedas separados por coma según su condición."""
    codigos = ""
    i = 0
    cantidad = len(listaMonedas)
    primero = True

    while i < cantidad:
        m = listaMonedas[i]
        codigo = m[0]
        tasa = m[2]

        cumple = False
        if condicion == "fuerte" and tasa > 1.0:
            cumple = True
        elif condicion == "igual" and tasa == 1.0:
            cumple = True
        elif condicion == "debil" and tasa < 1.0:
            cumple = True

        if cumple:
            if primero:
                codigos = str(codigo)
                primero = False
            else:
                codigos = codigos + ", " + str(codigo)

        i = i + 1

    return codigos

# Uso (generarReporteMonedasHtml):
codigosFuertes = obtenerCodigosPorClasificacion(monedasUnicas, "fuerte")
codigosIguales = obtenerCodigosPorClasificacion(monedasUnicas, "igual")
codigosDebiles = obtenerCodigosPorClasificacion(monedasUnicas, "debil")
```

3. Lista de validaciones estratégicas

Se implementaron 10 validaciones defensivas, verificadas contra el código real. Ninguna interrumpe abruptamente el programa: cada una informa al usuario, deja registro en la bitácora y regresa al menú.

N.º	Validación y función	Mecanismo técnico	Comportamiento defensivo
1	CSV de origen inexistente `datos.leerCsvPaises`	`try … except FileNotFoundError`	Informa el error, lo registra en la bitácora y retorna `None`.
2	CSV vacío o solo con encabezado `datos.leerCsvPaises`	`len(lineas) <= 1`	Detecta el archivo sin registros y no itera.
3	Fila con menos de 8 columnas `datos.validarFilaCsv`	`len(columnas) < 8`	Omite la fila e informa su número de línea.
4	Campos numéricos y nombre válidos `datos.validarFilaCsv`	`fs.esEnteroValido`, `fs.esFlotanteValido` y comparación con 0	Exige población, área y tasa mayores que 0 y nombre no vacío; descarta filas corruptas.
5	Archivos TOON inexistentes `datos.leerPaisesToon` / `leerTasasCambioToon`	`except FileNotFoundError`	Si se usan las opciones 2 a 7 antes de la 1, avisa «Primero ejecute la Opción 1», registra y vuelve al menú.
6	Opción del menú principal `main.main`	`fs.limpiarEspaciosExtremos` y `else`	Una opción fuera de 1–9 genera aviso, registro y nuevo ciclo del menú.
7	Letra de búsqueda (Opción 2) `main.solicitarLetraBusqueda`	`len(letra) == 1` y `fs.esLetraValida`	Rechaza números o palabras hasta recibir exactamente una letra.
8	Cantidad n de letras (Opción 4) `main.solicitarNLetrasMoneda`	`fs.esNumeroEnteroPositivo`	Exige un entero estrictamente positivo (n > 0); repite la solicitud.
9	Opción del submenú de bitácora `bitacora.ejecutarSubmenuBitacora`	`fs.aMayusculas` y comparación con A–E	Opción no reconocida: aviso y registro del error.
10	Errores de entrada/salida `datos.py`, `reportes.py`, `bitacora.py`	`try … except OSError`	Evita caídas por fallos de disco al guardar TOON, reportes o bitácora; informa y retorna `False`.

Además, `main.main` captura `KeyboardInterrupt` y `EOFError` para cerrar el programa de forma limpia y dejar el evento en la bitácora.

4. Estrategia de generación de los reportes

Estrategia general

La **Opción 1** lee `paises.csv` una sola vez, valida cada fila y guarda los datos crudos en dos archivos TOON: `paises.toon` (países) y `tasas_cambio.toon` (monedas únicas más la fecha de la tasa), para no repetir la misma moneda por cada país. La **Opción 2** reescribe `paises.toon` con los nombres normalizados. Las opciones de procesamiento y reporte (2 a 7) cargan siempre desde esos archivos, de modo que todos los resultados dependen de una única fuente. Como no se permiten diccionarios, cada país se modela como una **tupla** con posiciones fijas (nombre, capital, código, población, área, moneda, código de moneda, tasa) y la colección completa como una **lista de tuplas**.

Todos los reportes se construyen por concatenación de cadenas, con funciones pequeñas que devuelven un bloque de texto cada una; la función principal solo las une y las escribe con codificación UTF-8, dentro de un bloque `try/except OSError`. Los decimales pasan por `formatear2Decimales` y `formatear4Decimales` (`formatos.py`) para no depender de `round()`.

Reporte TXT de países (`reporte_paises.txt`)

Genera una ficha por país (`construirBloquePaisTxt`) con nombre y capital normalizados, código ISO, población, área, moneda, código de moneda, tasa por dólar, densidad y categoría, separadas por una línea en blanco: 35 fichas × 5 líneas = 175 líneas, más 8 líneas de resumen final, para un total de 183 líneas. Fragmento generado por el programa:

```
El país Argentina tiene como capital Buenos Aires, su código ISO es AR,
tiene 45808747 habitantes, un área de 2780400km2, su moneda es Peso Argentino,
código ARS, y hay 975.50 por cada dólar.
Densidad poblacional: 16.48 hab/km2. Categoría: Megaciudad.

El país Brasil tiene como capital Brasilia, su código ISO es BR,
tiene 214326223 habitantes, un área de 8515767km2, su moneda es Real,
código BRL, y hay 5.00 por cada dólar.
Densidad poblacional: 25.17 hab/km2. Categoría: Megaciudad.

El país Chile tiene como capital Santiago, su código ISO es CL,
...
==================================================
RESUMEN GENERAL:
Total de países: 35
Población total: 4540797546
Área total: 64933350 km2
Promedio de población: 129737072.74 habitantes
Promedio de área: 1855238.57 km2
==================================================
```

El resumen final cumple lo pedido en la Opción 5:

Indicador	Valor
Total de países	35
Población total	4540797546
Área total	64933350 km2
Promedio de población	129737072.74 habitantes
Promedio de área	1855238.57 km2

Reporte HTML de monedas (`reporte_monedas.html`)

Se arma con tres componentes: tarjeta de estadísticas generales (en etiquetas `<p>`), tabla detallada con la tasa a 4 decimales y su relación con el USD, y tabla de clasificación con colores alternados y celdas centradas. Una moneda es **Fuerte** si su tasa es mayor que 1, **Igual** si es 1 y **Débil** si es menor que 1, tal como define el enunciado («más fuerte = mayor tasa»). El reporte incluye una nota metodológica: en análisis financiero una cotización menor por dólar indica mayor valor unitario. Evidencia generada por el programa el 2026-10-14 01:25:02:

Estadística general
Cantidad total de monedas analizadas: 26
Tasa de cambio promedio: 456.7619
Moneda más fuerte (mayor tasa): Peso Colombiano (COP) con tasa 4150.0000
Moneda más débil (menor tasa): Libra Esterlina (GBP) con tasa 0.7600
Nota metodológica (Criterio económico): En economía financiera, una menor cotización nominal por dólar denota mayor apreciación unitaria (ej. GBP/CHF vs USD). Por instrucción de la consigna oficial, la clasificación principal se basa directamente en la magnitud nominal de la tasa.

Tabla detallada de monedas

Código	Nombre Normalizado	Tasa de Cambio (USD)	Relación con USD
ARS	Peso Argentino	975.5000	Fuerte
BRL	Real	5.0000	Fuerte
CLP	Peso Chileno	920.0000	Fuerte
COP	Peso Colombiano	4150.0000	Fuerte
EUR	Euro	0.9100	Débil
JPY	Yen Japones	148.2000	Fuerte
MXN	Peso mexicano	20.5000	Fuerte
PAB	Balboa Panameno	1.0000	Igual
PEN	Sol Peruano	3.7600	Fuerte
USD	Dolar estadounidense	1.0000	Igual
UYU	Peso Uruguayo	41.2000	Fuerte
GBP	Libra Esterlina	0.7600	Débil
CAD	Dolar canadiense	1.3500	Fuerte
AUD	Dolar Australiano	1.4800	Fuerte
CRC	Colon costarricense	500.0000	Fuerte
CNY	Yuan Chino	7.1000	Fuerte
INR	Rupia India	83.9000	Fuerte
KRW	Won Surcoreano	1335.0000	Fuerte
CHF	Franco Suizo	0.8500	Débil
EGP	Libra Egipcia	48.5000	Fuerte
ZAR	Rand Sudafricano	17.8000	Fuerte
ISK	Corona Islandesa	137.5000	Fuerte
BZD	Dolar Beliceno	2.0000	Fuerte

Código	Nombre Normalizado	Tasa de Cambio (USD)	Relación con USD
XOF	Franco CFA	600.0000	Fuerte
CDF	Franco Congoleno	2850.0000	Fuerte
STN	Dobra	22.5000	Fuerte

Tabla de clasificación

Clasificación	Cantidad de monedas	Códigos de las monedas
Fuerte	21	ARS, BRL, CLP, COP, JPY, MXN, PEN, UYU, CAD, AUD, CRC, CNY, INR, KRW, EGP, ZAR, ISK, BZD, XOF, CDF, STN
Igual	2	PAB, USD
Débil	3	EUR, GBP, CHF

Reporte HTML de densidad poblacional (`reporte_densidad.html`)

Calcula la densidad (población / área) de cada país, ordena manualmente con el método de burbuja y toma los 10 valores extremos para las dos tablas. La distribución de categorías se dibuja con un gráfico de barras vertical en ASCII dentro de una etiqueta `<pre>`, cuya altura se escala a la cantidad de países; los porcentajes se obtienen dividiendo cada categoría entre el total de países (35). Evidencia generada el 2026-10-14 01:25:02:

Estadísticas
Densidad Promedio Global: 683.55 hab/km²
Densidad Máxima: Monaco (19621.00 hab/km²)
Densidad Mínima: Australia (3.34 hab/km²)
• Megaciudades (> 10M): 23 países (65.71%)
• Ciudades Grandes (1M - 10M): 5 países (14.29%)
• Ciudades Medianas (100k - 999k): 4 países (11.43%)
• Ciudades Pequeñas (< 100k): 3 países (8.57%)

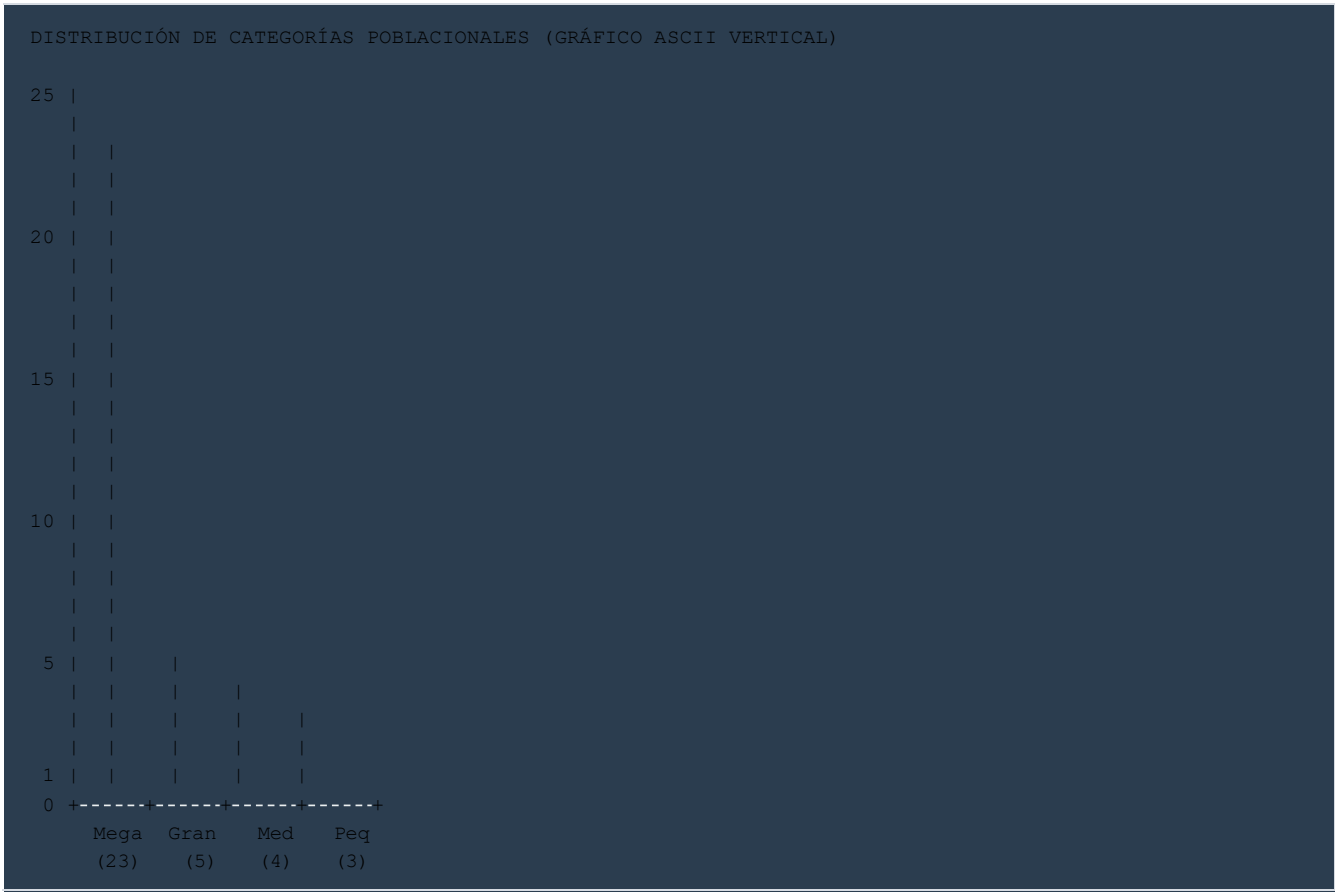
10 países con MAYOR densidad poblacional

#	País	ISO	Población	Área (km²)	Densidad (hab/km²)
1	Monaco	MC	39242	2	19621.00
2	Corea Del Sur	KR	51740000	100210	516.32
3	India	IN	1408044253	3287263	428.33
4	Japon	JP	125507000	377975	332.05
5	Reino Unido	GB	67330000	243610	276.38
6	Luxemburgo	LU	645397	2586	249.57
7	Liechtenstein	LI	39058	160	244.11
8	Alemania	DE	83200000	357022	233.04
9	Sao Tome Y Principe	ST	223364	964	231.71
10	Suiza	CH	8700000	41285	210.73

10 países con MENOR densidad poblacional

#	País	ISO	Población	Área (km²)	Densidad (hab/km²)
1	Australia	AU	25690000	7692024	3.34
2	Islandia	IS	372520	103000	3.62
3	Canada	CA	38250000	9984670	3.83
4	Argentina	AR	45808747	2780400	16.48
5	Belice	BZ	405272	22966	17.65
6	Uruguay	UY	3426260	176215	19.44
7	Brasil	BR	214326223	8515767	25.17
8	Chile	CL	19493184	756102	25.78
9	Peru	PE	33715471	1285216	26.23
10	Estados Unidos	US	331893745	9833517	33.75

Gráfico de distribución de categorías (ASCII vertical)



5. Reglamento de trabajo

Se adopta el reglamento de equipo propuesto por el curso (documento *Reglamento, Tarea Programada 1*). Las cinco reglas, con ejemplos de cómo se aplican en este proyecto, son:

Regla 1. Sin uso de IA.

El código debe ser creado sin uso de IA.

- Conocer cómo funciona cada función del código.
- Saber justificar por qué una función se hizo de cierta forma.
- Reutilizar código existente en lugar de crear funciones nuevas para todo.

Regla 2. Fechas de entrega.

Cumplir con las fechas de entrega.

- No atrasarse con las entregas de cada semana.
- Si algo tarda más, explicar la razón y cuánto se demorará.
- Realizar los commits a tiempo (al menos 3 por semana).

Regla 3. Informar las adiciones.

Informar las nuevas adiciones del código cuando son implementadas.

- Comunicar los cambios realizados.
- Informar qué opciones del menú ya están implementadas.
- Indicar si existen funciones que se puedan reutilizar más adelante.

Regla 4. Explicar el código.

Explicar cómo funciona el código a los miembros del equipo.

- Explicar cómo funciona cada función nueva al implementarla.
- Avisar cuando se reutiliza una función ya creada.
- Señalar nombres poco significativos para mejorarlos o documentarlos.

Regla 5. Cambios en código ajeno.

En caso de cambiar código de otra persona, consultar antes y comunicar el cambio.

- Acordar y comunicar los cambios de nombres poco significativos.
- Si una función falla, avisar a quien la creó y explicar la modificación.
- Proponer optimizaciones a quien la creó, verificando que no afecten sus cambios.

6. Agendas

Agenda 1

Código	Agenda 2026-S2-01
Reunión	1 - Arquitectura y consigna (Semana 1)
Fecha y hora	Viernes 25/09/2026, 18:00 - 19:30
Participante	Alexandra De Gracia

N.º	Tema a tratar	Duración	Responsable
1	Análisis de las restricciones pedagógicas del enunciado	20 min	Alexandra De Gracia
2	Arquitectura modular en 6 archivos	25 min	Alexandra De Gracia
3	Formato TOON y estructura del CSV	20 min	Alexandra De Gracia
4	Distribución de tareas y fechas de avance	25 min	Alexandra De Gracia

Agenda 2

Código	Agenda 2026-S2-02
Reunión	2 - Integración y cierre (Semana 3)
Fecha y hora	Sábado 10/10/2026, 19:00 - 21:00
Participante	Alexandra De Gracia

N.º	Tema a tratar	Duración	Responsable
1	Verificación de los cálculos matemáticos	30 min	Alexandra De Gracia
2	Revisión de los reportes TXT y HTML	30 min	Alexandra De Gracia
3	Bitácora y exportación a CSV	30 min	Alexandra De Gracia
4	Consolidación de la documentación final y PSP	30 min	Alexandra De Gracia

7. Minutas

Minuta 1 - Reunión del 25/09/2026 (Agenda 2026-S2-01)

Asistente: Alexandra De Gracia. **Hora:** 18:00 - 19:30.

N.º	Acuerdo	Responsable
1	Dividir el sistema en módulos independientes: main, datos, formateo_strings, calculos, reportes y bitacora.	Alexandra De Gracia
2	Implementar la manipulación de cadenas carácter por carácter en `formateo_strings.py`, sin `.replace()`.	Alexandra De Gracia
3	Serializar el TOON con sangría de 4 y 8 espacios en `datos.py`.	Alexandra De Gracia
4	Diseñar el menú interactivo y las firmas de las funciones de cálculo.	Alexandra De Gracia
5	Entregar el Avance 1 (cadenas, CSV y TOON) el 02/10/2026.	Alexandra De Gracia

Próxima reunión: sábado 10/10/2026.

Minuta 2 - Reunión del 10/10/2026 (Agenda 2026-S2-02)

Asistente: Alexandra De Gracia. **Hora:** 19:00 - 21:00.

N.º	Acuerdo	Responsable
1	Ratificar los cálculos de población (total, promedio y mediana), de área y densidad, y de la tasa de cambio promedio.	Alexandra De Gracia
2	Eliminar el código muerto de `reportes.py` (commit `713c9cf`) y estandarizar los nombres a camelCase (commit `9b3c3ac`).	Alexandra De Gracia
3	Dejar la bitácora en `bitacora.bin` con exportación a `bitacora.csv`.	Alexandra De Gracia
4	Completar la documentación, la plantilla de bitácora de problemas y el PDF para la entrega del 16/10/2026.	Alexandra De Gracia

8. Cronograma

Plan de tres semanas, del 25 de septiembre al 16 de octubre de 2026, con los hitos de entrega.

Fecha	Actividad	Responsable	Tiempo
25/09	Análisis del enunciado y de las restricciones; definición de la arquitectura modular.	Alexandra De Gracia	3 h
26/09 - 28/09	`formateo_strings.py`: mayúsculas, espacios, tildes, paréntesis, guiones bajos y capitalización carácter por carácter.	Alexandra De Gracia	6 h
29/09 - 01/10	`datos.py`: lectura y validación del CSV y serialización TOON.	Alexandra De Gracia	7 h
02/10	**Hito: Avance 1.** Pruebas y commit del avance.	Alexandra De Gracia	2 h
03/10 - 04/10	`calculos.py`: población, mediana con burbuja, área, densidad y categorías.	Alexandra De Gracia	6 h
05/10 - 06/10	Procesamiento de monedas, redondeo manual y soporte UTF-8.	Alexandra De Gracia	6 h
07/10 - 08/10	`reportes.py`: reporte TXT y reporte HTML de monedas.	Alexandra De Gracia	6 h
09/10	**Hito: Avance 2.** Reporte HTML de densidad con gráfico ASCII y limpieza de código muerto.	Alexandra De Gracia	5 h
09/10 - 10/10	`bitacora.py`: registro, submenú de búsqueda y exportación a CSV.	Alexandra De Gracia	4 h
10/10 - 12/10	Pruebas generales del sistema y corrección de errores.	Alexandra De Gracia	7 h
13/10 - 15/10	Ajustes finales (bitácora binaria con `pickle`, separación de `paises.toon` y `tasas_cambio.toon`), documentación final, estadística de tiempos y exportación a PDF.	Alexandra De Gracia	8 h
16/10	**Hito: Entrega final** en TEC Digital antes de las 11:40 pm.	Alexandra De Gracia	-
	TOTAL		**60 h**

9. Conclusiones del trabajo

Problemas encontrados y soluciones

Registrados según la plantilla *Plantilla_Bitacora_ResoluciónProb* del curso.

N.º	Día del problema	Descripción del problema	Día de solución	Descripción de la solución	Horas
1	26/09/2026	Prohibición estricta de `.replace()` y métodos automáticos de cadenas para normalizar nombres y textos.	28/09/2026	Funciones manuales carácter por carácter con ciclos `while` en `formateo_strings.py` (mayúsculas, tildes, caracteres especiales, paréntesis, guiones bajos y capitalización).	5
2	27/09/2026	Prohibición del uso de diccionarios para almacenar países y monedas.	29/09/2026	Los registros se modelaron como listas de tuplas con posiciones fijas.	4
3	29/09/2026	Serialización y lectura manual del formato TOON con sangría de 4 y 8 espacios y guiones delimitadores.	01/10/2026	Funciones manuales en `datos.py` que escriben y leen el TOON línea por línea, sin bibliotecas.	6
4	02/10/2026	Cálculo de la mediana sin `statistics` ni funciones de ordenamiento nativas.	04/10/2026	Ordenamiento de burbuja manual en `calculos.py` y selección del valor central (`n // 2`).	3
5	03/10/2026	Ambigüedad en «moneda más fuerte»: criterio económico (menor tasa) contra el criterio literal de la Opción 4 (mayor tasa).	05/10/2026	Se usó el criterio literal del enunciado en la Opción 4 y en el HTML, y se documentó la diferencia con una nota metodológica.	4
6	04/10/2026	Código muerto residual (asignación sobrescrita de inmediato) en `construirTablaDetalladaMonedas`.	09/10/2026	Se eliminó la asignación residual (commit `713c9cf`).	1
7	05/10/2026	`UnicodeEncodeError` en Windows al escribir tildes y símbolos como km².	06/10/2026	Se forzó `encoding='utf-8'` al abrir archivos y se normalizaron las tildes en las salidas.	3
8	05/10/2026	Redondeo y formato a 2 y 4 decimales sin `round()` ni formato automático.	06/10/2026	Redondeo con potencias de 10 y relleno de ceros con cadenas (hoy en `formatos.py`).	4
9	14/10/2026	La bitácora se guardaba como texto plano con «|» y no era un archivo binario real.	14/10/2026	Se migró `bitacora.bin` a `pickle` : una tupla (fecha, descripción) por registro, en modo `ab` , leída hasta `EOFError` .	2
10	14/10/2026	Los datos de monedas se repetían por país (p. ej. el EUR varias veces) y la «fecha de la tasa» era la fecha del día.	14/10/2026	Se separó la persistencia en `paises.toon` y `tasas_cambio.toon` (monedas únicas con `fecha_tasa` guardada en el archivo).	2

Aprendizajes obtenidos

Lecciones aprendidas durante el desarrollo de la tarea: 8 de carácter personal y 8 de carácter técnico.

Lecciones personales

Código	Lección	Descripción
LP-01	Disciplina y avance semanal	Dividir el trabajo en avances semanales evitó la acumulación de tareas al final y permitió detectar errores a tiempo.
LP-02	Comunicación técnica temprana	Explicar el diseño y las decisiones antes de programar, aunque sea por escrito, ayudó a encontrar fallos antes de implementarlos.
LP-03	Honestidad y ética ante las restricciones	Respetar las prohibiciones del enunciado (`.replace()` , diccionarios, `round()`) aun cuando había atajos obligó a entender el problema a fondo.
LP-04	Resiliencia ante problemas de bajo nivel	Resolver tareas que normalmente hace una librería requirió paciencia y probar paso a paso sin desanimarse.
LP-05	Coordinación con Git	Hacer commits pequeños y frecuentes con mensajes claros dio un historial entendible y permitió volver atrás sin perder trabajo.
LP-06	Atención al detalle	Revisar los casos de frontera (listas vacías, filas incompletas, valores cero) evitó errores que no aparecen con datos normales.
LP-07	Autonomía e investigación	Buscar por cuenta propia cómo funcionan el formato TOON, el HTML y los archivos binarios desarrolló la capacidad de aprender sin depender de ejemplos.
LP-08	Cultura de refactorización	Dedicar tiempo a limpiar el código (nombres, funciones cortas, sin duplicados) hizo más fácil mantenerlo y explicarlo en la defensa.

Lecciones técnicas

Código	Lección	Descripción
LT-01	Texto carácter por carácter	Recorrer cadenas con ciclos `while` y comparar cada carácter permite implementar mayúsculas, eliminación de tildes y capitalización sin métodos automáticos.
LT-02	Generación de HTML5	Construir HTML por concatenación de cadenas, con estilos en línea y un gráfico ASCII dentro de `<pre>`, produce reportes legibles en cualquier navegador.
LT-03	Ordenamiento de burbuja	Implementar el ordenamiento manualmente permitió calcular la mediana y los rankings sin `sort()` y comprender su costo.
LT-04	Serialización TOON	Escribir y leer el formato con sangrías de 4 y 8 espacios exige controlar con precisión la jerarquía del archivo.
LT-05	Persistencia binaria	Guardar la bitácora con `pickle` en modo append, con tuplas (fecha y hora, descripción), conserva el historial entre ejecuciones.
LT-06	Redondeo manual	Redondear con potencias de 10 y rellenar ceros con cadenas permite formatear 2 y 4 decimales sin `round()` .
LT-07	Eliminación de malos olores	Detectar código muerto, duplicado y métodos largos, y corregirlos con funciones pequeñas y reutilizables, mejora la calidad del programa.
LT-08	Validación defensiva	Validar la existencia de archivos, las columnas del CSV y las entradas del usuario evita caídas y deja registro de cada error en la bitácora.

Conclusión del trabajo

La tarea permitió integrar archivos, cadenas, tuplas, listas y operaciones numéricas en un sistema completo que lee, normaliza, calcula y reporta datos de países y monedas, cumpliendo las restricciones del curso (sin diccionarios, sin `.replace()` y con cálculos manuales). En lo técnico se ganó experiencia con TOON, HTML, CSV y archivos binarios, y en la eliminación de malos olores. En lo personal se reforzaron la planificación semanal, la investigación autónoma y el uso de GitHub como herramienta de trabajo.

10. Estadística de tiempos (Personal Software Process - PSP)

El **Personal Software Process (PSP)** es un método desarrollado por Watts Humphrey en el Software Engineering Institute de la Universidad Carnegie Mellon. Propone que cada programador registre el tiempo que dedica a cada actividad y los defectos que encuentra, para estimar mejor trabajos futuros y mejorar su proceso. Registrar las horas por actividad permite medir el esfuerzo real, compararlo con lo planificado y usarlo como base para estimar tareas similares.

Actividad realizada	Horas estimadas	Horas reales	Desviación	% del esfuerzo real
Análisis de requerimientos y lectura del enunciado	2	2.5	+0.5	4.6 %
Distribución de tareas y planificación	2	2	0	3.7 %
Investigación (TOON, HTML, archivos binarios, PSP)	5	5.5	+0.5	10.2 %
Programación	22	26	+4	48.1 %
Documentación interna (docstrings y comentarios)	3	3.5	+0.5	6.5 %
Pruebas y validaciones de ejecución	3	4	+1	7.4 %
Elaboración de la documentación del proyecto	6	7	+1	13.0 %
Limpieza de código y refactorización (olores)	3	3.5	+0.5	6.5 %
TOTAL	**46**	**54**	**+8**	**100 %**

La programación concentra el 48.1 % del esfuerzo (26 de 54 horas); el análisis, la planificación y la investigación suman 10 horas (18.5 %), y las pruebas y la limpieza de código, 7.5 horas (13.9 %). El esfuerzo real superó en 8 horas (17.4 %) lo estimado, sobre todo en programación (+4 h). Para una próxima tarea similar conviene reservar más tiempo para programación y pruebas, ya que varios errores (UTF-8, redondeo, código muerto, bitácora binaria) se detectaron en la etapa final.